

How Onam finds the paths that matter

From cloud inventory to a confirmed, MITRE-mapped, scored attack path — the method, end to end.

7 clouds

One property graph

Confirmed edges only

MITRE ATT&CK

Choke points

FAIR-style risk

THE ARGUMENT, IN ORDER

00	Executive summary	A finding is a fact.
01	The problem with a list	A modern multi-cloud estate generates findings faster than any team can trace them.
02	Ingestion — 7 clouds	Posture scanning connects through read-only cloud roles.
03	One data model — the property graph	Everything normalises into a single per-tenant property graph.
04	Search — BFS from entry to crown jewel	Search is breadth-first from entry points toward crown jewels...
05	Verification — confirmed edges only	A path that exists in the graph is a hypothesis, not a finding.
06	MITRE mapping, choke points, and risk	Each verified hop maps to a MITRE ATT&CK technique...
07	Summary	One claim, repeated at every stage: a finding is a fact, and only a verified route is a decision.

READS WITH [02 Cloud risk in dollars](#) · [03 One graph, one data model](#) · [04 Security & trust](#)

00 Executive summary

A finding is a fact. A verified path is a decision.

Every cloud scanner produces thousands of findings. The hard problem is not detection — it is knowing which handful of them combine into a route an attacker could actually walk to something that matters. Onam treats that as a graph-reasoning problem, solved the same way every time.

This paper describes the method end to end: how Onam inventories 7 clouds, normalises everything into one property graph, derives the relationships an attacker would exploit from evidence, searches for routes from entry points to crown jewels, and walks only the edges that evidence confirms. Each hop is mapped to a MITRE ATT&CK tactic and technique with a remediation step; the paths are scored on likelihood and impact; the choke points are marked; and findings on attack paths have their FAIR-style loss estimate raised, so the output reads in business terms rather than as a severity label.

The claim, precisely. Onam does not promise to block attacks at runtime. It promises something more useful for prioritisation: a reproducible, evidence-based map of the routes that exist in your cloud, rebuilt on every scan, with a deterministic identity for every finding so results are stable across rescans.

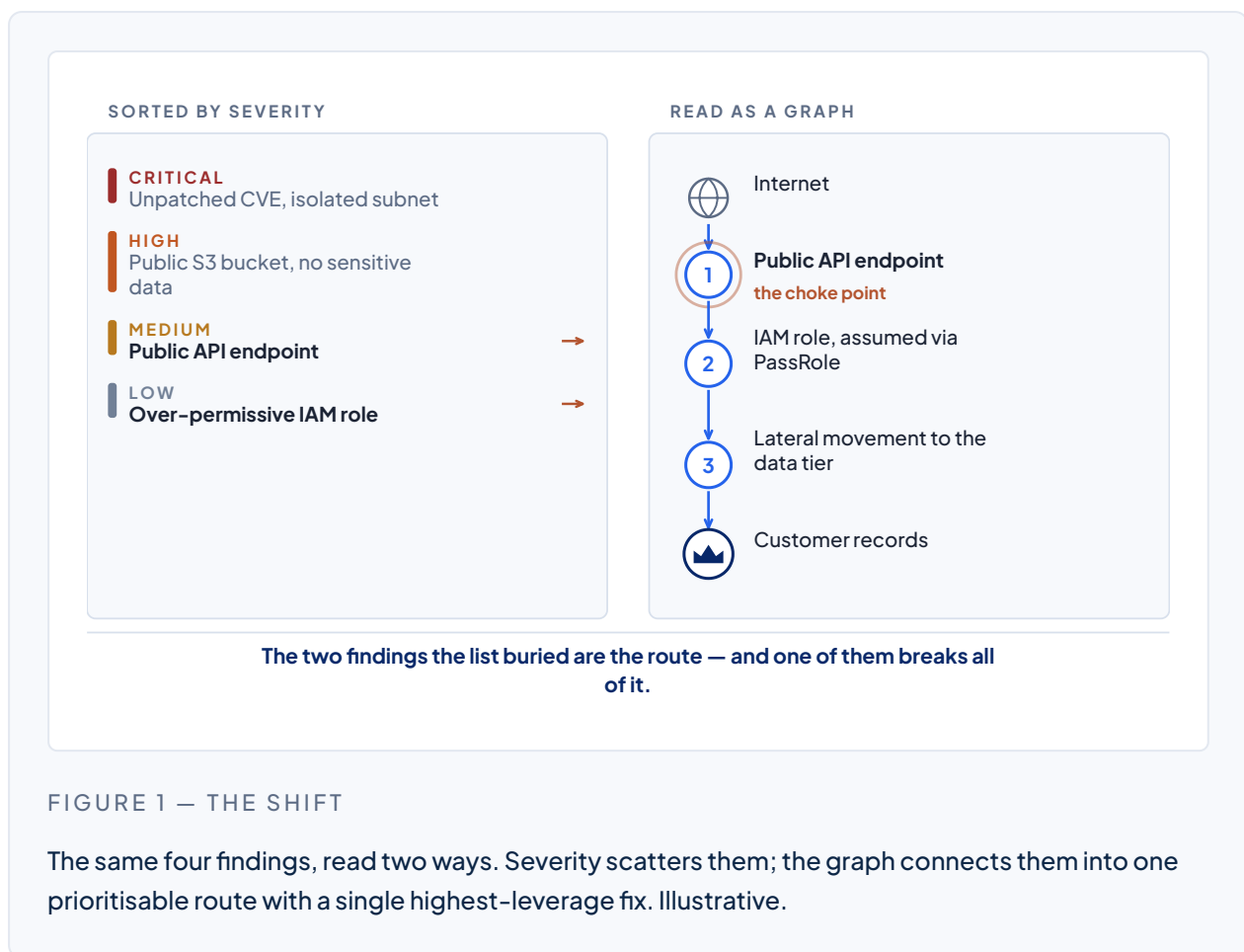
01 The problem with a list

A modern multi-cloud estate generates findings faster than any team can trace them. The industry's default answer is severity: sort by Critical, work down the list. But severity is a property of a finding **in isolation**. It cannot tell you that a "medium" public endpoint, a "low" over-permissive role and an unencrypted data store are — together — a single four-hop route to your customer records.

Attackers do not read your list. They chain. A route is only as strong as its weakest link, and the link that matters is rarely the one with the scariest CVSS score. To prioritise like a defender who thinks like an attacker, you have to reason over the relationships between findings, not the findings themselves.

02 Ingestion — 7 clouds

Posture scanning connects through read-only cloud roles; agentless workload scanning runs inside your account.



The method starts with telemetry that is safe to collect. Posture scanning connects to each cloud with the lowest privilege that permits a security read. On AWS Onam stores no secret — it assumes your role for short-lived credentials; where a cloud needs a stored credential, it is kept in AWS Secrets Manager, encrypted with AWS KMS. Agentless workload scanning is a separate mechanism that runs inside your account with its own permissions.

Cloud	Connection	Privilege posture
AWS	Cross-account IAM role via sts:AssumeRole, unique External ID per tenant	SecurityAudit + ReadOnlyAccess; STS credentials ≤ 60 min
Azure	Entra service principal	Reader + read-only Graph
GCP	Service account or keyless Workload Identity Federation	6 read-only roles
OCI	IAM user + signing key	inspect + read only
Alibaba / IBM	RAM read-only · Service ID	Read-only, exchanged for a short-lived Bearer token
Kubernetes	Read-only ServiceAccount	get / list / watch

Full detail of the trust model is in Whitepaper 04.

03 One data model — the property graph

Everything normalises into a single per-tenant property graph. Assets are nodes; edges are typed — containment, attachment, IAM trust and network reachability. Catalog-driven derives then add the *abuse* edges: the relationships an attacker would actually use, as distinct from the ones the cloud provider documents.

This is the step that makes cross-cloud reasoning possible at all. A path that starts in one provider and ends in another is only visible when both are described in the same vocabulary. Architecture detail is in Whitepaper 03.

04 Search — BFS from entry to crown jewel

Search is breadth-first from entry points — the internet, admin and other identities, CI/CD, third parties and other clouds — toward crown jewels: the data stores, keys and other high-value resources a path can end at. Breadth-first matters: it finds the *shortest* route first, and likelihood decays with every hop, so shorter routes score higher.

05 Verification — confirmed edges only

A path that exists in the graph is a hypothesis, not a finding. Every edge carries a verdict from the evidence behind it — posture, identity, network, data and threat-detection signals — and is marked confirmed, blocked or a gap. Search walks only confirmed edges.

Why this gate exists. An unverified path is worse than no path: it spends a security team's credibility on a route that may not exist. Verification is the difference between a graph that suggests and a graph that asserts.

06 MITRE mapping, choke points, and risk

Each verified hop maps to a MITRE ATT&CK technique, so the route reads as an attack narrative rather than a list of resource IDs, and each hop carries a remediation step. Paths are scored on likelihood and impact: a WAF or MFA confirmed on the route lowers the score, a high-EPSS vulnerability on a node raises it. The choke point — the node appearing on the most confirmed routes — is marked, with the number of paths fixing it would block.

Finally, findings that sit on attack paths have their FAIR-style loss estimate raised, so the paths carry business weight in the risk view. Method detail is in Whitepaper 02.

07 Summary

One claim, repeated at every stage: a finding is a fact, and only a verified route is a decision.

- Severity ranks findings against each other; a graph ranks routes against reality.
- 7 clouds into one property graph with typed abuse edges, built from evidence.
- Breadth-first search from entry points to crown jewels.
- Only edges confirmed by evidence are walked.
- MITRE mapping for narrative, choke points for leverage, a FAIR-style estimate for business weight.
- Onam does not block at runtime. It tells you, reproducibly, which few fixes matter most.

Cloud risk in business terms

How Onam puts a FAIR-style loss estimate on each finding — with every input shown, so a board can see what drives the number.

FAIR-style

EPSS × exposure

Sensitivity ×

Regulatory ×

Min · likely · max

THE ARGUMENT, IN ORDER

00	Executive summary	Severity tells you what is broken.
01	The FAIR model, briefly	The model is not ours.
02	The inputs	Every input is shown, and the ones that describe your business are yours to set.
03	From finding to estimate	The estimate sits on the finding, and attack paths raise it.
04	From one finding to a queue	A remediation queue you can argue with is worth more than one you have to trust.
05	Summary	Every number in this paper can be taken apart — that is the point of it.

00 Executive summary

Severity tells you what is broken. Business impact tells you what to fix first.

Security teams are drowning in "criticals". Boards are not asking how many — they are asking how much. This paper explains how Onam puts a FAIR-style loss estimate on each finding, so the conversation can move from a spreadsheet of severities to a ranked list in business terms.

FAIR — Factor Analysis of Information Risk — is published by The Open Group as the O-RT (Risk Taxonomy) and O-RA (Risk Analysis) standards. It decomposes risk into how often a loss happens and how large it is when it does. Onam applies that structure to every finding its engines raise, using data the platform already holds and inputs you can set, and shows the calculation next to the result.

The claim, precisely. Onam does not run a full FAIR analysis with simulation. It computes a FAIR-style estimate per finding — a minimum, likely and maximum figure — from stated inputs, so findings can be compared in one unit. It is an ordering tool, not a forecast.

01 The FAIR model, briefly

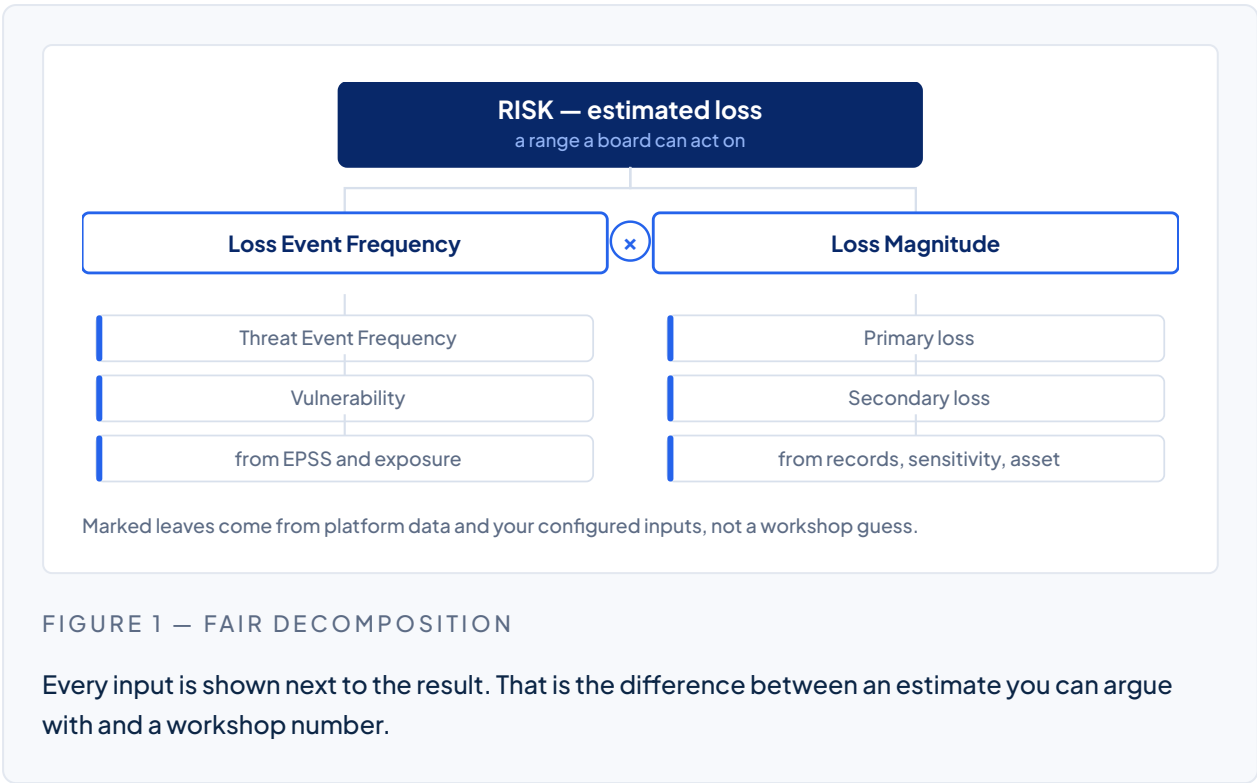
The model is not ours. FAIR is an open, published standard.

FAIR expresses risk as the product of how often a loss happens and how large it is when it does. Onam keeps that structure and fills each factor from platform data.

The strength of the structure is that each factor is inspectable. When Onam scores a finding, a reviewer can open the calculation and see which input drove the number — not a black-box score between 0 and 100.

02 The inputs

Every input is shown, and the ones that describe your business are yours to set.



A dollar figure is only as credible as the numbers behind it. These are the inputs Onam uses for each finding.

Input	Source / basis	Effect
Exploit likelihood	FIRST EPSS score for the vulnerability	Starting point for how often a loss event is expected
Exposure	Whether the resource is publicly reachable; attack-path signals	A public resource counts fully, an internal one at a fraction; findings on attack paths are raised
Records at risk and per-record cost	Defaults you can replace with your own figures	Base loss magnitude
Data-sensitivity weight	Onam data classification (DSPM)	Restricted ×3.0 → Confidential ×2.0 → Internal ×1.0 → Public ×0.1
Asset weight	Resource type; crown-jewel type from attack-path analysis	Keys, admin roles, data warehouses and similar weigh far more than ordinary resources
Regulatory weight and fines	Regulations that apply by region and data class (GDPR, HIPAA, PCI DSS, CCPA, SOX and others)	Highest applicable weight, not stacked; a separate fine range from each regulation's penalty structure, your record count and revenue

Inputs that describe your business — record counts, per-record cost, revenue, sensitivity weights — are configurable per tenant. Change one and the figure moves, which is the point of an inspectable model.

The honest limit. A quantified figure is an estimate, not an invoice. It is built on stated assumptions and defaults; it does not predict what a specific breach will cost. Anyone presenting it as a guarantee is misusing it, including us.

03 From finding to estimate

The estimate sits on the finding, and attack paths raise it.

1. **Find.** Engines raise findings across 7 clouds — posture, identity, data, vulnerability, threat detection.
2. **Connect.** Attack-path analysis marks findings that sit on a route to a crown jewel, and how many routes they sit on.
3. **Estimate.** Each finding gets a minimum, likely and maximum loss figure and a risk tier from the inputs above, with exposure raised for findings on attack paths.
4. **Explain.** The calculation inputs and the MITRE ATT&CK techniques involved are stored with the result.

04 From one finding to a queue

A remediation queue you can argue with is worth more than one you have to trust.

Ordered by estimated loss, the findings list becomes a queue in business terms: the largest contributors first, each with the inputs that put it there. Combined with attack-path choke points — the resource that appears on the most routes — it shows where one fix removes the most risk.

Because the same findings from the same 9,853 CSPM posture rules feed both the compliance and the risk views, a control gap and a high-loss finding are two readings of one underlying fact — not two systems to reconcile.

05 Summary

Every number in this paper can be taken apart — that is the point of it.

- FAIR is an open standard from The Open Group; Onam applies a FAIR-style estimate, not a full simulation.
- Likelihood starts from EPSS and exposure; magnitude from records, sensitivity, asset value and regulation.
- Inputs that describe your business are yours to set, and every input is shown with the result.
- Findings on attack paths are raised, so chained risk carries more weight than isolated risk.
- The output is a range with stated assumptions, not a prediction of a specific loss.

One graph, one data model

Why every engine writes the same finding contract into one store — and what that makes possible that a drawer of tools cannot.

One canonical store

Deterministic finding IDs

Per-tenant property graph

Typed abuse edges

29 engines

THE ARGUMENT, IN ORDER

00	Executive summary	Consolidation stops being a slide when it becomes a data model.
01	Ingestion into a normalised inventory	Two tables come out of ingestion, and every capability in the platform depends on them.
02	The canonical finding contract	Consolidation lives or dies on the schema.
03	The property graph	The cloud documents its relationships.
04	Engines as writers, not silos	29 engines write into the same store.
05	What this architecture buys	Cross-cloud attack paths.
06	Summary	Adding a capability adds a writer to the same store — never another silo.

READS WITH [01 Attack-path methodology](#)

00 Executive summary

Consolidation stops being a slide when it becomes a data model.

Most security platforms are a drawer of tools that each keep their own findings and their own view of the world. Onam is built the opposite way: every engine, for every cloud, writes the same finding contract into one shared store, and everything reasons over one property graph. This paper explains that design and why it is the thing that makes cross-cloud attack paths, risk in business terms and painless extensibility possible.

The architectural claim is specific and testable: **there are no per-engine report tables to reconcile**. A gateway reads only from one canonical findings store; a per-tenant graph turns those findings and the assets they touch into nodes and typed edges. Adding a capability does not add a silo — it adds another writer to the same store.

Why it matters to a buyer. Posture, identity, data, workloads, SaaS and runtime detection share one schema, so a finding in one domain can form part of an attack path that ends in another — which is impossible when each tool hoards its own results.

01 Ingestion into a normalised inventory

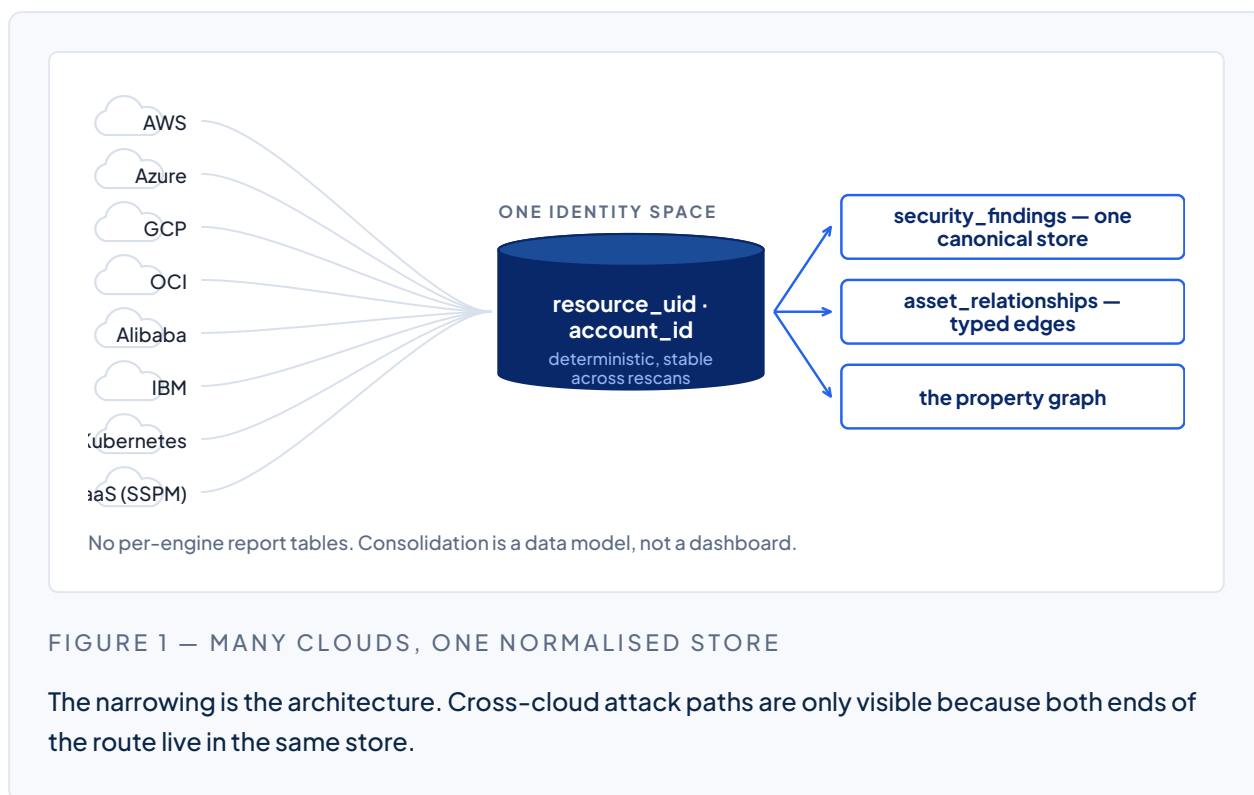
Two tables come out of ingestion, and every capability in the platform depends on them.

Posture scanning connects through read-only cloud roles; agentless workload scanning runs inside your account (Whitepaper 01, §02). The pipeline runs four stages — enumeration → enrichment → normalisation → drift — and writes two things every capability depends on: an asset inventory and an asset-relationship table.

7 clouds plus SaaS collapse into one normalised identity space, covering 549 cloud services and 11,433 rule definitions.

02 The canonical finding contract

Consolidation lives or dies on the schema.



Onam requires every engine to emit the same standardised finding, keyed consistently, so one table can hold results from posture, identity, data, workload, code and SaaS engines without translation.

Field	Canonicalises	Why it matters
resource_uid	ARN vs OCID vs self-link	One identity for an asset across clouds
account_id	account vs subscription vs project	Ownership normalised across providers
finding_id	deterministic hash	Same issue → same ID on every rescan; stable history
tenant_id	not-null isolation key	Every row is tenant-scoped by construction
provider_metadata	provider-specific detail	Nothing is lost; specifics travel with the finding

Because `finding_id` is deterministic, the same issue keeps the same ID on every rescan, so history, trends and remediation status line up without a human reconciling IDs by hand.

03 The property graph

The cloud documents its relationships. The graph adds the ones an attacker would use.

On top of the findings and relationships, Onam builds a per-tenant property graph. Assets are nodes; edges are typed — containment, attachment, IAM trust and network reachability — and catalog-driven derivers add the abuse edges an attacker would use.

The derivers are the important part. A cloud provider documents the relationships it intends; an attacker uses the ones that exist. The graph has to hold both.

04 Engines as writers, not silos

29 engines write into the same store. That is the extensibility claim: a new capability is a new writer, not a new database, a new schema and a new reconciliation job. It is also why a posture finding and a runtime detection can appear on the same path — they were never in separate systems to begin with.

The honest limit. One shared schema is a constraint as well as a gift. An engine whose output does not fit the contract has to be modelled to fit it, and that is real work. We take that cost deliberately, because the alternative — per-engine tables — is the thing that makes cross-domain reasoning impossible later.

05 What this architecture buys

- **Cross-cloud attack paths.** A route from one provider to another is only visible when both are described in one vocabulary.
- **Risk in business terms.** A FAIR-style estimate needs consistent asset identity and data classification to compute magnitude; the shared schema supplies both.
- **Stable history.** Deterministic IDs mean trend lines and remediation tracking are real, not approximate.
- **Extensibility.** Adding an engine adds coverage, not another silo to reconcile.

06 Summary

Adding a capability adds a writer to the same store — never another silo.

- One canonical findings store. No per-engine report tables.
- A standardised finding contract with deterministic IDs and mandatory tenant scoping.
- A per-tenant property graph with typed edges, plus derived abuse edges.
- 29 engines as writers into the same model — extensibility without silos.
- The constraint is real and accepted: fitting the contract is work, and worth it.

Compliance, mapped once

How Onam evaluates a control once and reports it against 78 frameworks — and reads each gap alongside the attack paths it sits on.

One rule → many controls

Rules as code

78 frameworks

Evidence per control

Gap → attack path

THE ARGUMENT, IN ORDER

00	Executive summary	Evaluate the control once.
01	One finding, many controls	Frameworks disagree about wording far more than they disagree about controls.
02	Rules as code — why coverage extends itself	A new rule extends every framework it maps to, and a new framework inherits every rule already written.
03	Frameworks supported	Coverage here means continuous evaluation, not an audit export generated the week before.
04	Compliance meets the attack path	Two control failures of equal audit weight are not equal risk.
05	What framework count does and does not prove	A large framework number proves breadth of mapping.
06	Summary	Fix the condition once; every framework that maps to it updates itself.

00 Executive summary

Evaluate the control once. Report it against every framework.

Compliance work is mostly duplication: the same control, re-checked and re-evidenced for PCI, then SOC 2, then ISO, then a customer questionnaire. Onam removes the duplication by decoupling the check from the framework. Rules are evaluated once as code; a single finding maps to many controls across 78 frameworks at once.

This paper explains the mapping model, why rules-as-code makes framework coverage extend automatically, and how compliance and attack-path reasoning reinforce each other — so a control gap is not just a failed check but, where it applies, a step on an attack path.

An honest framing. Framework count is a coverage feature, not a differentiator on its own — several platforms publish large numbers. Onam's distinct value is the architecture that produces the mapping (one rule → many controls, auto-extending) and the ability to read a compliance gap alongside the attack paths it sits on. This paper leads with the mechanism, not the number.

01 One finding, many controls

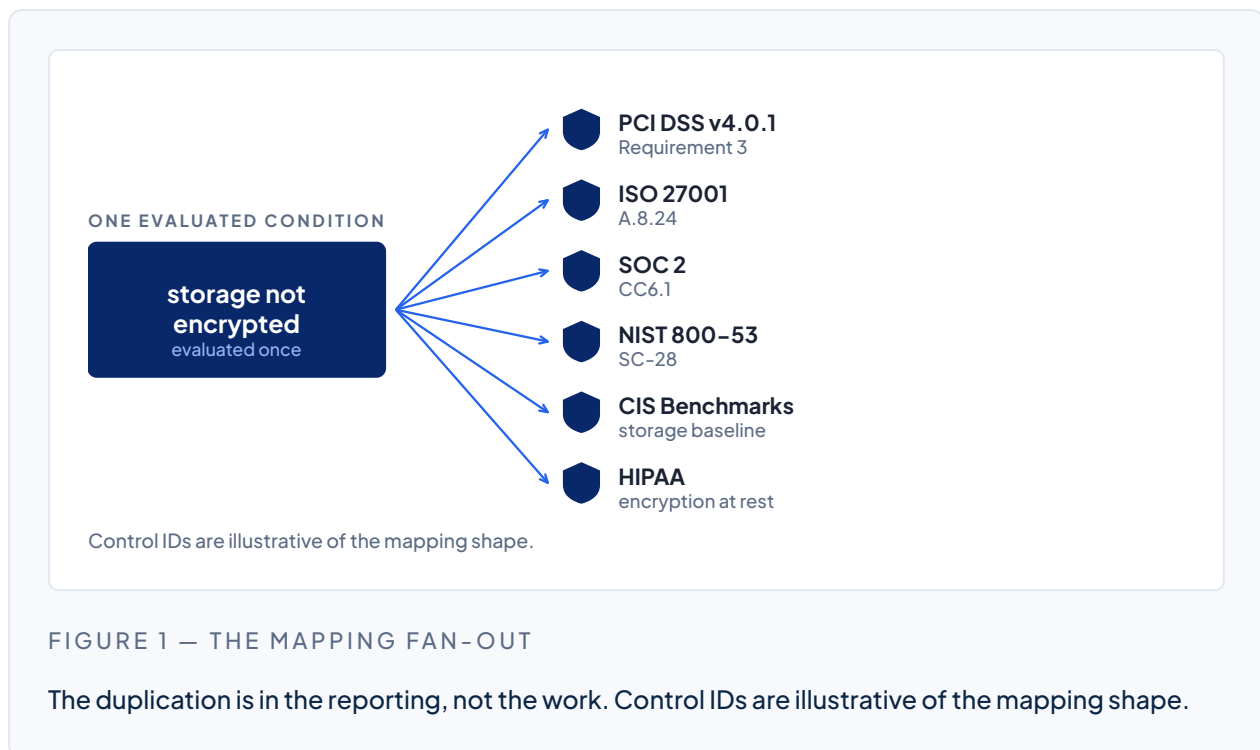
Frameworks disagree about wording far more than they disagree about controls.

A framework is, at bottom, a set of controls. Many controls across many frameworks ask for the same underlying thing — "encrypt data at rest", "restrict public access", "enforce MFA". Onam evaluates that underlying condition once and maps the result to every control it satisfies or violates.

02 Rules as code — why coverage extends itself

A new rule extends every framework it maps to, and a new framework inherits every rule already written.

Onam's rules are versioned YAML the platform loads and evaluates; nothing is hardcoded. This is what makes framework coverage compound rather than accumulate by hand. When a new rule is added, it automatically extends every framework whose controls it maps to. When a new framework is onboarded, its controls map to existing rule IDs — no new checks to write.



1. **Add a rule.** A new YAML rule ships; every mapped framework gains coverage for the condition it checks.
2. **Add a framework.** A new framework's controls map to existing rule IDs; coverage appears without new detection code.
3. **Add a rule of your own.** A rule written for an internal standard joins the same library, and its framework mappings decide which reports it appears in.

The compounding effect. Because checks and frameworks are decoupled, work done for one obligation pays down others automatically. A rule written to satisfy an internal standard may also close a PCI and an ISO control the same day — without anyone re-authoring the check.

03 Frameworks supported

Coverage here means continuous evaluation, not an audit export generated the week before.

Onam supports 78 frameworks out of the box across regulatory, industry and best-practice standards — for example PCI DSS v4.0.1, SOC 2, ISO 27001:2022, HIPAA, GDPR, NIST 800-53, NIST 800-171, FedRAMP Moderate and High, and CIS benchmarks. Coverage is delivered as continuous evaluation, not a point-in-time audit export.

The mapping rests on a catalog of 11,433 rule definitions across 7 clouds and 549 cloud services, of which 9,853 are CSPM posture rules.

04 Compliance meets the attack path

Two control failures of equal audit weight are not equal risk.

A failed control is a compliance fact. The same failing condition is also a node in the property graph — and Onam records on each resource whether it sits on an attack path and whether it is a choke point — so a failed control can be read alongside whether its failure lies on a confirmed route to a crown jewel. That reframes remediation: two "failures" of equal audit weight are not equal if one sits on a choke point and the other reaches nothing.

Lens	Question it answers	What it ranks by
Compliance	Which controls are failing, and where is the evidence?	Framework obligation
Risk	Which failure is on a reachable path to something that matters?	Attack-path signals and FAIR-style loss estimate

Read together, the two lenses turn an audit backlog into an ordered queue: the control gaps that are also choke points come first, because fixing one of them closes an obligation *and* removes a route.

05 What framework count does and does not prove

A large framework number proves breadth of mapping. It does not prove depth of evaluation, quality of evidence, or that the controls most relevant to your obligations are covered well.

- **It does prove** that a platform has invested in mapping, and that onboarding a new obligation is unlikely to require new detection work.
- **It does not prove** that any individual control is checked the way your auditor expects, nor that evidence export matches your reporting format.
- **The question to ask any vendor**, including us: show me the rule behind this control, and the evidence it produces.

06 Summary

Fix the condition once; every framework that maps to it updates itself.

- One evaluated condition maps to many controls across 78 frameworks — fix once, update everywhere.
- Rules-as-code means coverage extends automatically when either a rule or a framework is added.
- A control gap that sits on an attack path is not the same as one that reaches nothing — and the risk view raises the finding behind it.
- Framework count is a coverage feature. The architecture that produces the mapping is the differentiator.